

Task 1: Build a modified kernel

Step 1: Start MINIX and back up the kernel

Approach:

I started MINIX in QEMU and logged in as root. Before making any kernel changes, I created a backup of the kernel source using tar:

```
tar cvf safe.tar /usr/src/kernel
```

This archived the kernel source so I would have a copy to return to if my changes broke the system.

```
a usr/src/kernel/arch/i386/io_intr.S
a usr/src/kernel/arch/i386/io_inw.S
a usr/src/kernel/arch/i386/io_outb.S
a usr/src/kernel/arch/i386/io_outl.S
a usr/src/kernel/arch/i386/io_outw.S
a usr/src/kernel/arch/i386/kernel.lds
a usr/src/kernel/arch/i386/klib.S
a usr/src/kernel/arch/i386/klib16.S
a usr/src/kernel/arch/i386/memory.c
a usr/src/kernel/arch/i386/mpx.S
a usr/src/kernel/arch/i386/multiboot.S
a usr/src/kernel/arch/i386/multiboot.h
a usr/src/kernel/arch/i386/oxpcie.c
a usr/src/kernel/arch/i386/oxpcie.h
a usr/src/kernel/arch/i386/pre_init.c
a usr/src/kernel/arch/i386/protect.c
a usr/src/kernel/arch/i386/proto.h
a usr/src/kernel/arch/i386/sconst.h
a usr/src/kernel/arch/i386/serial.h
a usr/src/kernel/arch/i386/include/arch_clock.h
a usr/src/kernel/arch/i386/include/arch_watchdog.h
a usr/src/kernel/arch/i386/include/archconst.h
a usr/src/kernel/arch/i386/include/hw_intr.h
```

I also referred to the usage, monitor, boot, shutdown, and tar documentation while doing the lab.

Problems Encountered:

N/A.

Solution:

N/A.

Lessons Learned:

I learned that tar is used for backups, usage explains MINIX configuration and make hdboot, monitor explains the boot monitor, boot explains the startup process, and shutdown explains how to safely shut down or reboot the system.

Step 2: Locate the idle code

Approach:

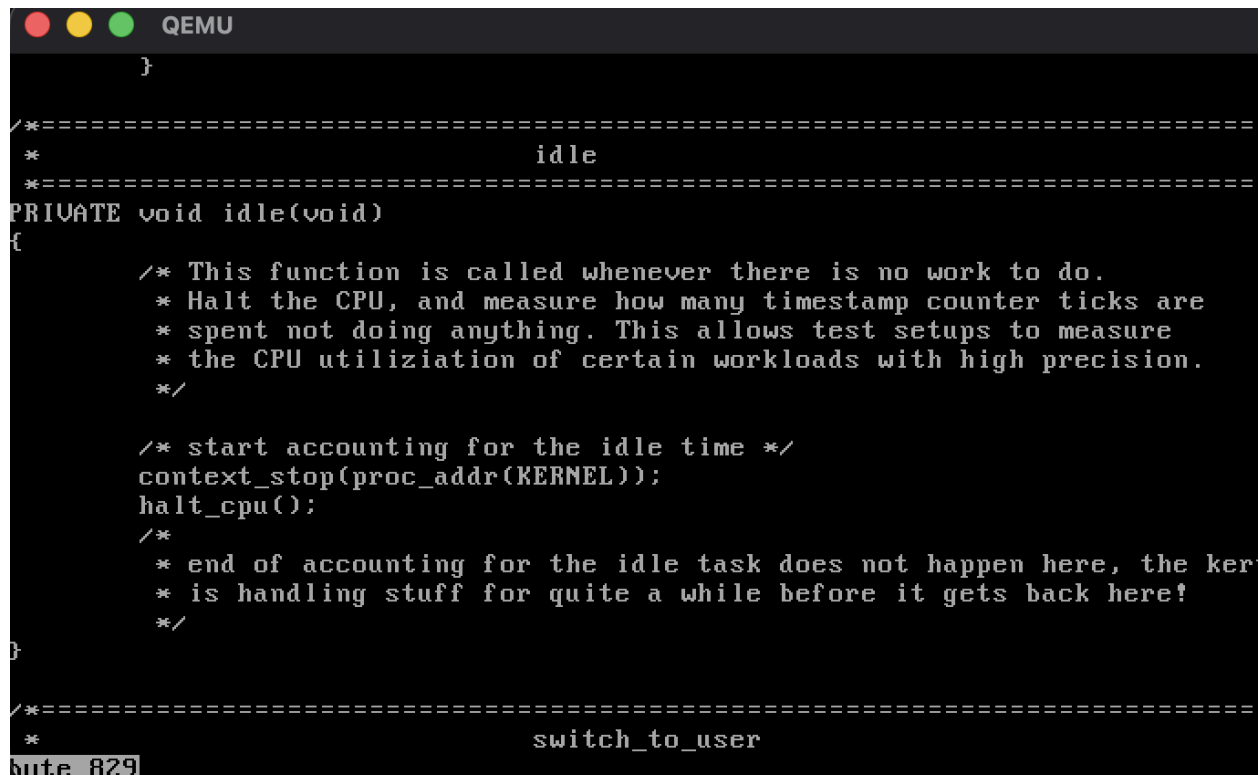
I needed to modify the kernel so that an @ appears when the system is idle. Since the kernel source is in /usr/src, I went into the kernel directory and searched for idle-related code:

```
cd /usr/src/kernel
grep -n "idle" *.c
```

This showed that idle() was defined in proc.c, so I viewed that section of the file:

```
sed -n '80,145p' proc.c | more
```

The comment inside idle() said the function is called whenever there is no work to do, so I knew this was the right location for the modification.



```

}

/*=====
 *
 *=====
PRIVATE void idle(void)
{
    /* This function is called whenever there is no work to do.
     * Halt the CPU, and measure how many timestamp counter ticks are
     * spent not doing anything. This allows test setups to measure
     * the CPU utilization of certain workloads with high precision.
     */

    /* start accounting for the idle time */
    context_stop(proc_addr(KERNEL));
    halt_cpu();
    /*
     * end of accounting for the idle task does not happen here, the ker
     * is handling stuff for quite a while before it gets back here!
     */
}

/*=====
 *
 *=====
switch_to_user
bute 829

```

Problems Encountered:

The QEMU screen was small, so it was hard to read long command output.

Solution:

I used *more* to page through output more easily.

Lessons Learned:

I learned that the proc.c file has important process and scheduling code, and that idle() is called when there are no runnable processes.

Step 3: Edit proc.c

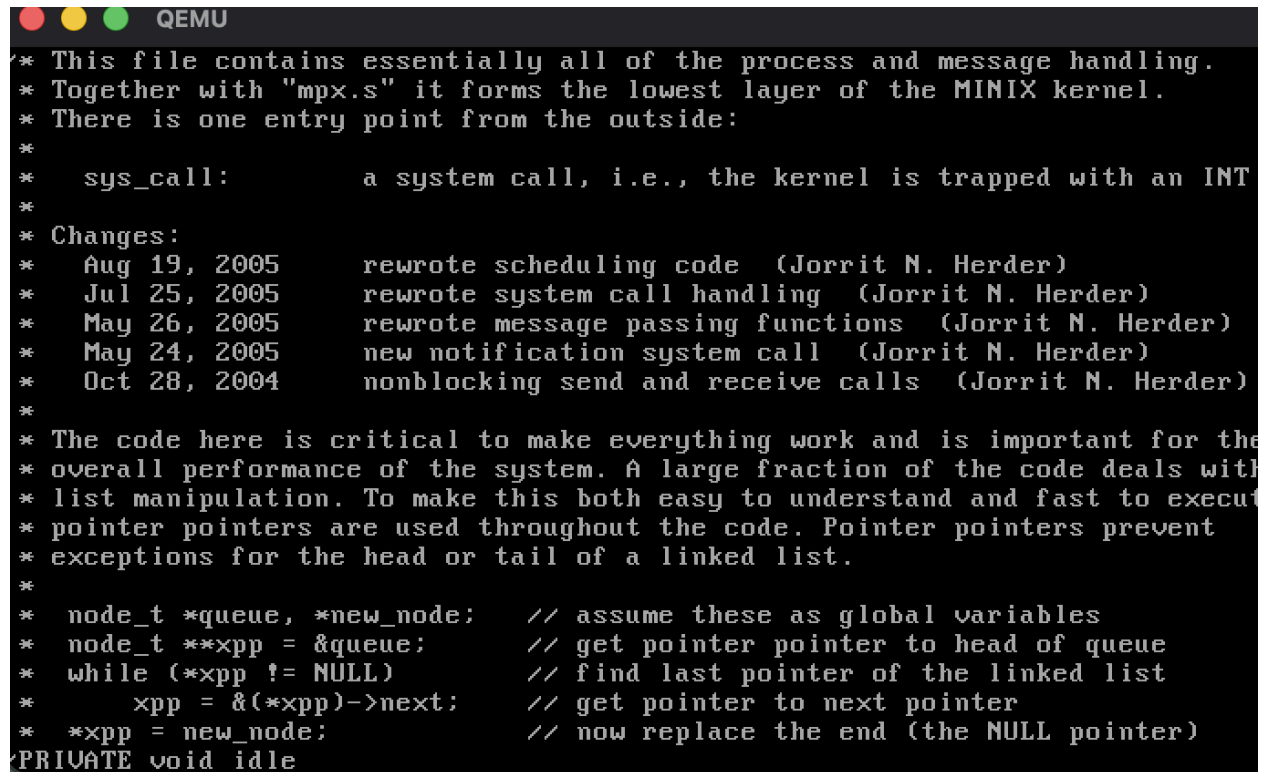
Approach:

I opened proc.c in vi:

vi proc.c

Then I searched for the idle function inside vi with:

/PRIVATE void idle



```

* This file contains essentially all of the process and message handling.
* Together with "mpx.s" it forms the lowest layer of the MINIX kernel.
* There is one entry point from the outside:
*
* sys_call:      a system call, i.e., the kernel is trapped with an INT
*
* Changes:
*   Aug 19, 2005    rewrote scheduling code (Jorrit N. Herder)
*   Jul 25, 2005    rewrote system call handling (Jorrit N. Herder)
*   May 26, 2005    rewrote message passing functions (Jorrit N. Herder)
*   May 24, 2005    new notification system call (Jorrit N. Herder)
*   Oct 28, 2004    nonblocking send and receive calls (Jorrit N. Herder)
*
* The code here is critical to make everything work and is important for the
* overall performance of the system. A large fraction of the code deals with
* list manipulation. To make this both easy to understand and fast to execute
* pointer pointers are used throughout the code. Pointer pointers prevent
* exceptions for the head or tail of a linked list.
*
* node_t *queue, *new_node;    // assume these as global variables
* node_t **xpp = &queue;      // get pointer pointer to head of queue
* while (*xpp != NULL)         // find last pointer of the linked list
*     xpp = &(*xpp)->next;     // get pointer to next pointer
* *xpp = new_node;             // now replace the end (the NULL pointer)
PRIVATE void idle

```

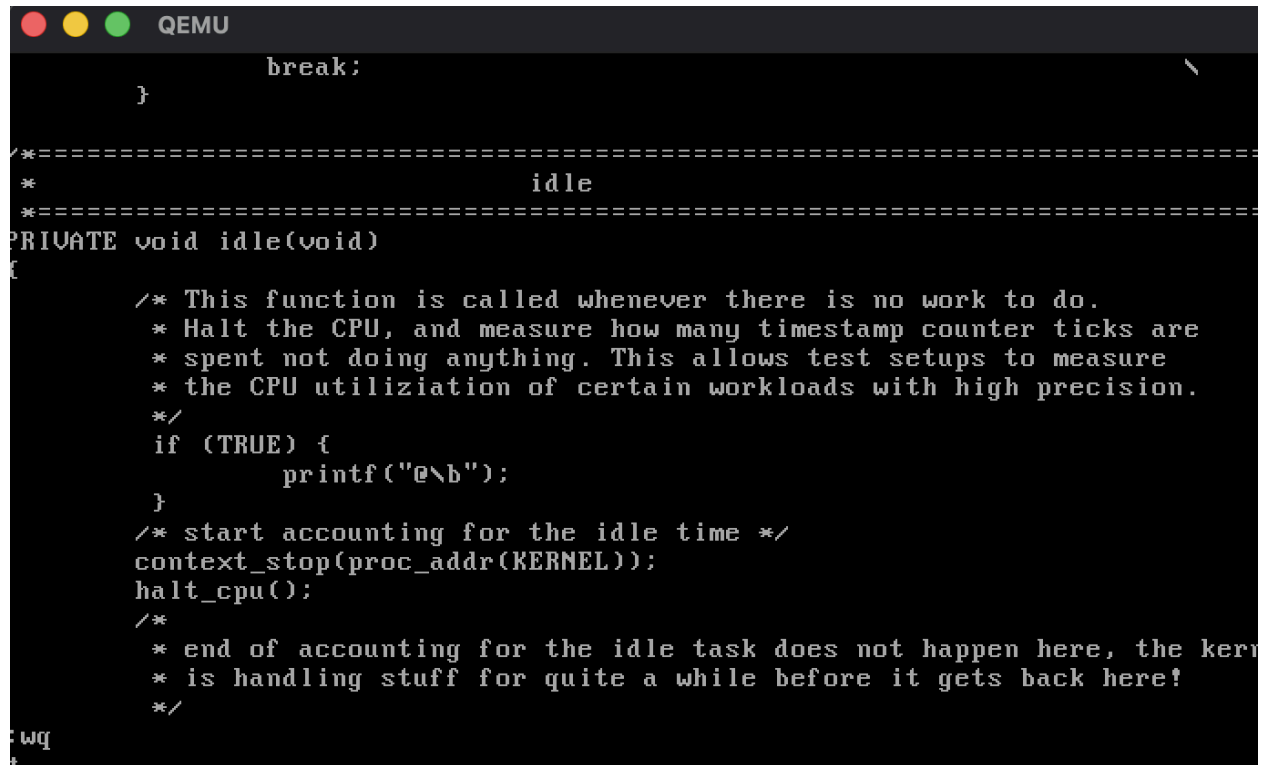
Inside idle(), I added the following block before the idle accounting and CPU halt code:

```

if (TRUE) {
    printf("@\b");
}

```

The @ displays the symbol, and \b moves the cursor back over it so the cursor appears on top of the @.



```

break;
}

/*=====
*                               idle
*=====
PRIVATE void idle(void)
{
    /* This function is called whenever there is no work to do.
     * Halt the CPU, and measure how many timestamp counter ticks are
     * spent not doing anything. This allows test setups to measure
     * the CPU utilization of certain workloads with high precision.
     */
    if (TRUE) {
        printf("@\b");
    }
    /* start accounting for the idle time */
    context_stop(proc_addr(KERNEL));
    halt_cpu();
    /*
     * end of accounting for the idle task does not happen here, the kern
     * is handling stuff for quite a while before it gets back here!
     */
}

```

Problems Encountered:

Editing in vi was difficult because I had to be careful not to change the other kernel code.

Solution:

After saving, I checked the function again to make sure the change was in the correct place.

Lessons Learned:

I learned that `context_stop()` is used for accounting, while `halt_cpu()` is what actually stops the CPU until an interrupt occurs. Because of that, I placed the print statement before those lines.

Step 4: Rebuild and install the modified kernel

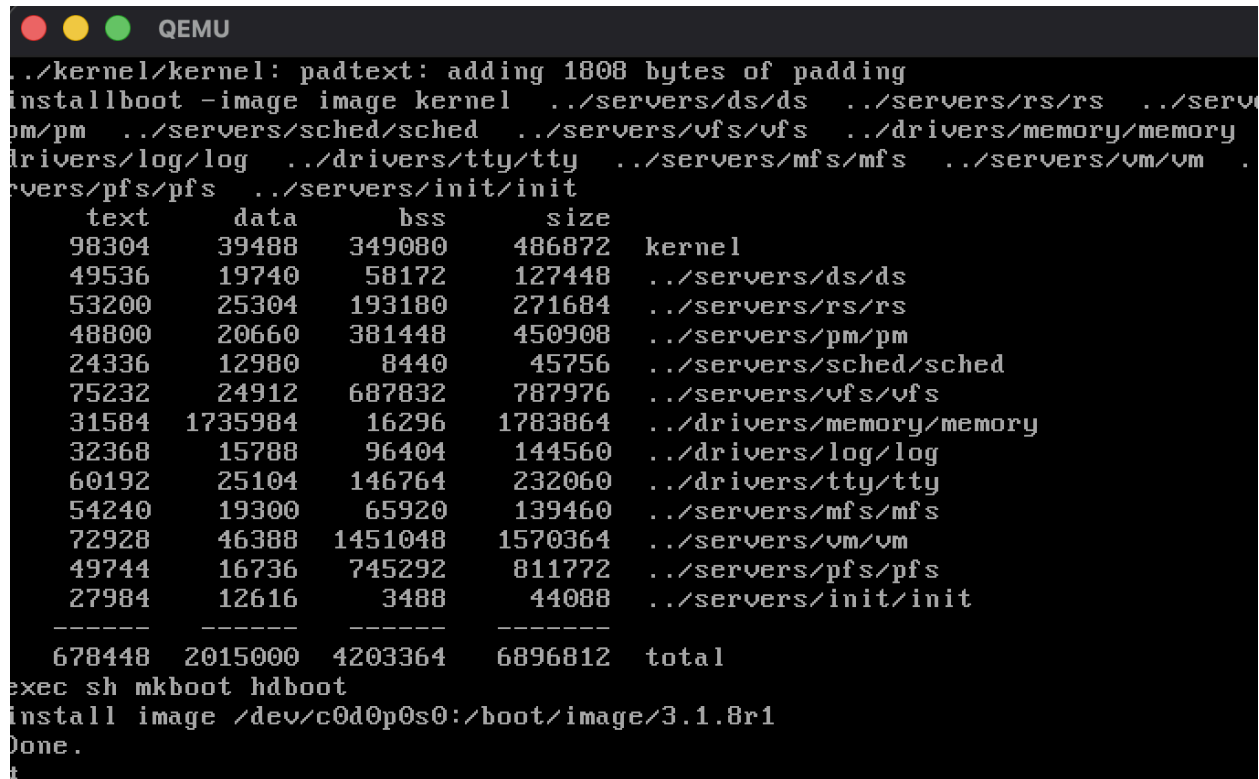
Approach:

After editing the source code, I rebuilt and installed the kernel image from /usr/src/tools:

```
cd /usr/src/tools
```

```
make hdboot
```

This compiled the kernel and installed the new boot image.



```

QEMU
./kernel/kernel: padtext: adding 1808 bytes of padding
installboot -image image kernel ../servers/ds/ds ../servers/rs/rs ../serv
pm/pm ../servers/sched/sched ../servers/vfs/vfs ../drivers/memory/memory
drivers/log/log ../drivers/tty/tty ../servers/mfs/mfs ../servers/vm/vm .
servers/pfs/pfs ../servers/init/init
      text      data      bss      size
98304      39488      349080      486872  kernel
49536      19740       58172      127448  ../servers/ds/ds
53200      25304      193180      271684  ../servers/rs/rs
48800      20660      381448      450908  ../servers/pm/pm
24336      12980       8440       45756  ../servers/sched/sched
75232      24912      687832      787976  ../servers/vfs/vfs
31584     1735984       16296     1783864  ../drivers/memory/memory
32368      15788       96404      144560  ../drivers/log/log
60192      25104      146764      232060  ../drivers/tty/tty
54240      19300      65920      139460  ../servers/mfs/mfs
72928      46388     1451048     1570364  ../servers/vm/vm
49744      16736      745292      811772  ../servers/pfs/pfs
27984      12616       3488       44088  ../servers/init/init
-----
678448     2015000     4203364     6896812  total
exec sh mkboot hdboot
install image /dev/c0d0p0s0:/boot/image/3.1.8r1
Done.
#

```

Problems Encountered:

N/A.

Solution:

N/A.

Lessons Learned:

I learned that make hdboot builds the image and installs it as a bootable MINIX image.

Step 5: Reboot into the modified kernel

Approach:

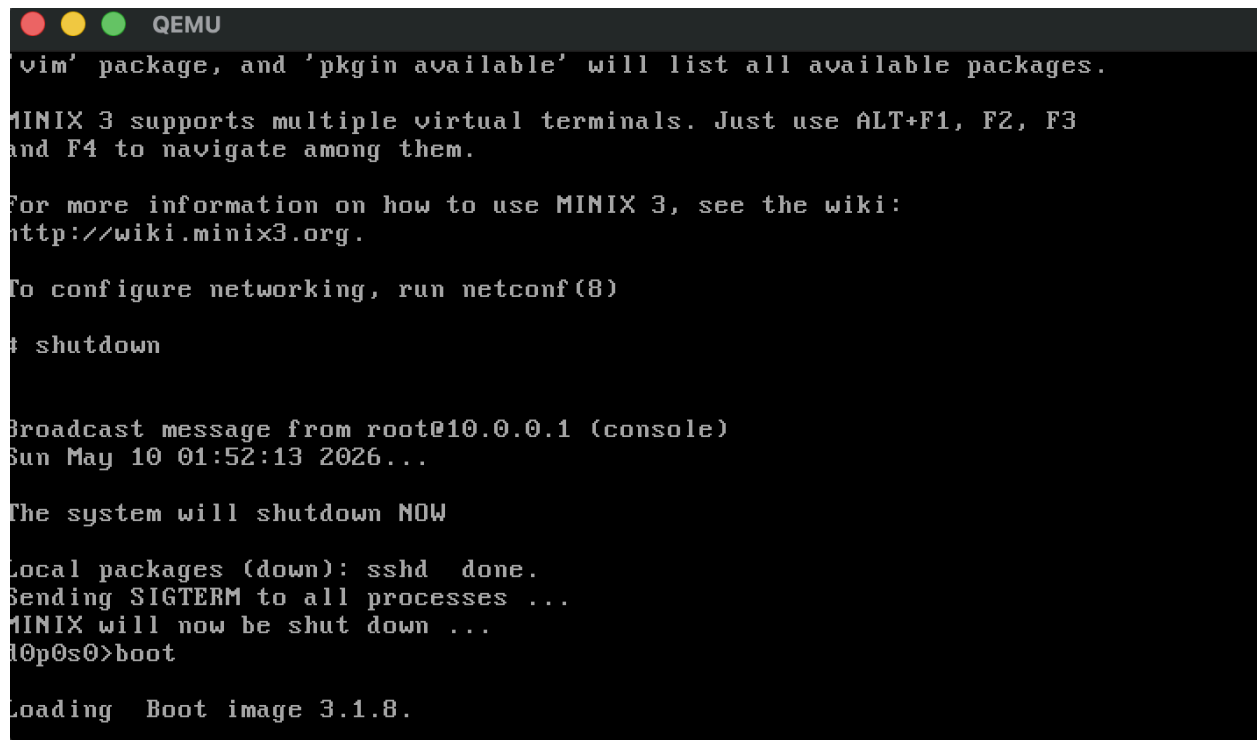
After *make hdboot* finished, I rebooted MINIX with:

```
shutdown -R now
```

This performed a hardware-style reset, which worked better in QEMU than manually using the boot monitor's boot command.

Problems Encountered:

When I used the boot monitor command boot, QEMU sometimes got stuck at Loading Boot image image.



```
QEMU
vim' package, and 'pkgin available' will list all available packages.
MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3
and F4 to navigate among them.
For more information on how to use MINIX 3, see the wiki:
http://wiki.minix3.org.
To configure networking, run netconf(8)
# shutdown

Broadcast message from root@10.0.0.1 (console)
Sun May 10 01:52:13 2026...

The system will shutdown NOW

Local packages (down): sshd done.
Sending SIGTERM to all processes ...
MINIX will now be shut down ...
l0p0s0>boot

Loading Boot image 3.1.8.
```

*note: The time is inconsistent because I forgot to take a screenshot of the issue earlier.

Solution:

I used *shutdown -R now* for rebooting after the rebuild. When the boot monitor got stuck, I quit QEMU from the Mac terminal and restarted it.

Lessons Learned:

I learned that MINIX shutdown matters because it flushes filesystem changes safely, and that *shutdown -R now* was more reliable in my QEMU setup.

Task 2: Test it

Step 6: Test the modification

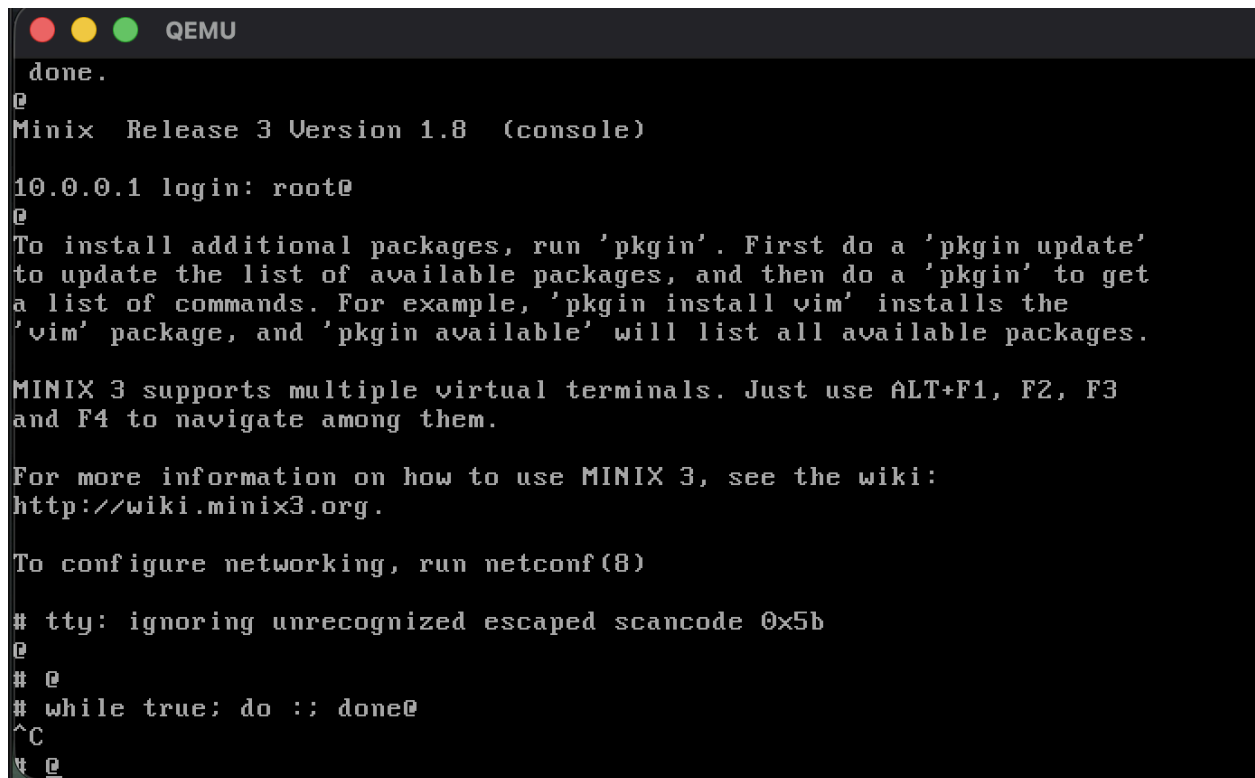
Approach:

After booting the modified kernel, I logged in as root and saw the @ appear near the cursor when the system was idle. To test that the symbol only appeared when the system was idle, I ran a busy loop:

```
while true; do ;; done
```

Then I stopped it with:

Ctrl+C



```
done.
@
Minix  Release 3 Version 1.8  (console)

10.0.0.1 login: root@
@
To install additional packages, run 'pkgin'. First do a 'pkgin update'
to update the list of available packages, and then do a 'pkgin' to get
a list of commands. For example, 'pkgin install vim' installs the
'vim' package, and 'pkgin available' will list all available packages.

MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3
and F4 to navigate among them.

For more information on how to use MINIX 3, see the wiki:
http://wiki.minix3.org.

To configure networking, run netconf(8)

# tty: ignoring unrecognized escaped scancode 0x5b
@
# @
# while true; do ;; done@
^C
# @
```

The loop kept the shell busy. While it was running, the @ stopped. After I pressed *Ctrl+C*, the system returned to idle and the @ appeared again.

Problems Encountered:

At first, I was not sure how to create a simple busy-waiting test in the MINIX shell. I needed a command that would keep the CPU busy without printing a lot of output to the screen, so I looked up how shell infinite loops and no-op commands work.

Solution:

I used the busy loop:

```
while true; do ;; done
```

I learned that true can be used as a command that always succeeds, so while true creates an infinite loop. I also learned that : is the shell null utility, meaning it does nothing but still counts

as a valid command inside the loop. This let me keep the shell busy without producing output. I stopped the loop with *Ctrl+C*.

References:

POSIX : null utility: <https://www.unix.com/man-page/posix/1p/colon>

ShellCheck explanation of empty loops/no-op commands:

<https://www.shellcheck.net/wiki/SC1060>

true command: <https://linuxcommandlibrary.com/man/true>

Lessons Learned:

I learned that a busy-waiting loop can be created by repeatedly running a command that does nothing. This gave me a clear comparison between a busy system and an idle system. When the busy loop was running, the @ output stopped, and when I stopped the loop, the @ returned.

Step 7: Return to the old kernel

Approach:

After testing, I wanted to stop using the modified kernel because the @ output was distracting. I shut down to the boot monitor:

```
shutdown
```

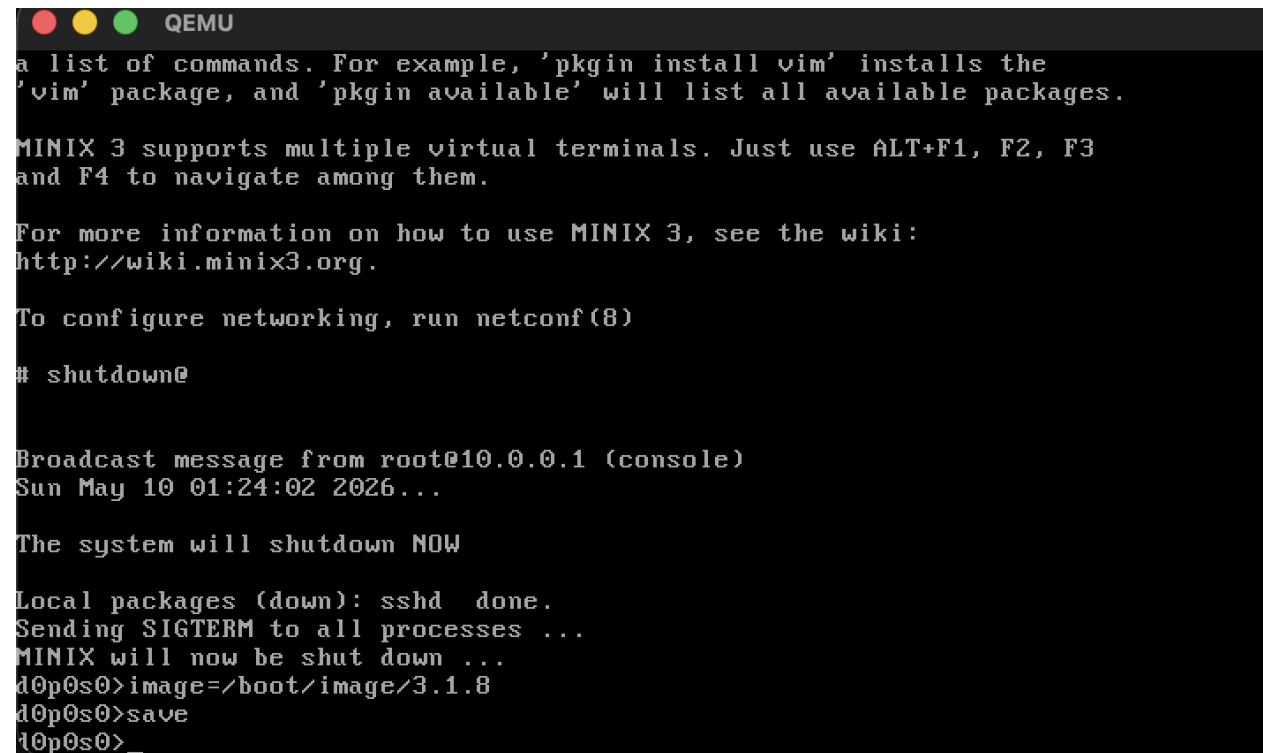
Then I listed the available boot images:

```
ls /boot/image
```

I selected the old stable image and saved it as the default:

```
image=/boot/image/3.1.8
```

```
save
```



```
QEMU
a list of commands. For example, 'pkgin install vim' installs the
'vim' package, and 'pkgin available' will list all available packages.

MINIX 3 supports multiple virtual terminals. Just use ALT+F1, F2, F3
and F4 to navigate among them.

For more information on how to use MINIX 3, see the wiki:
http://wiki.minix3.org.

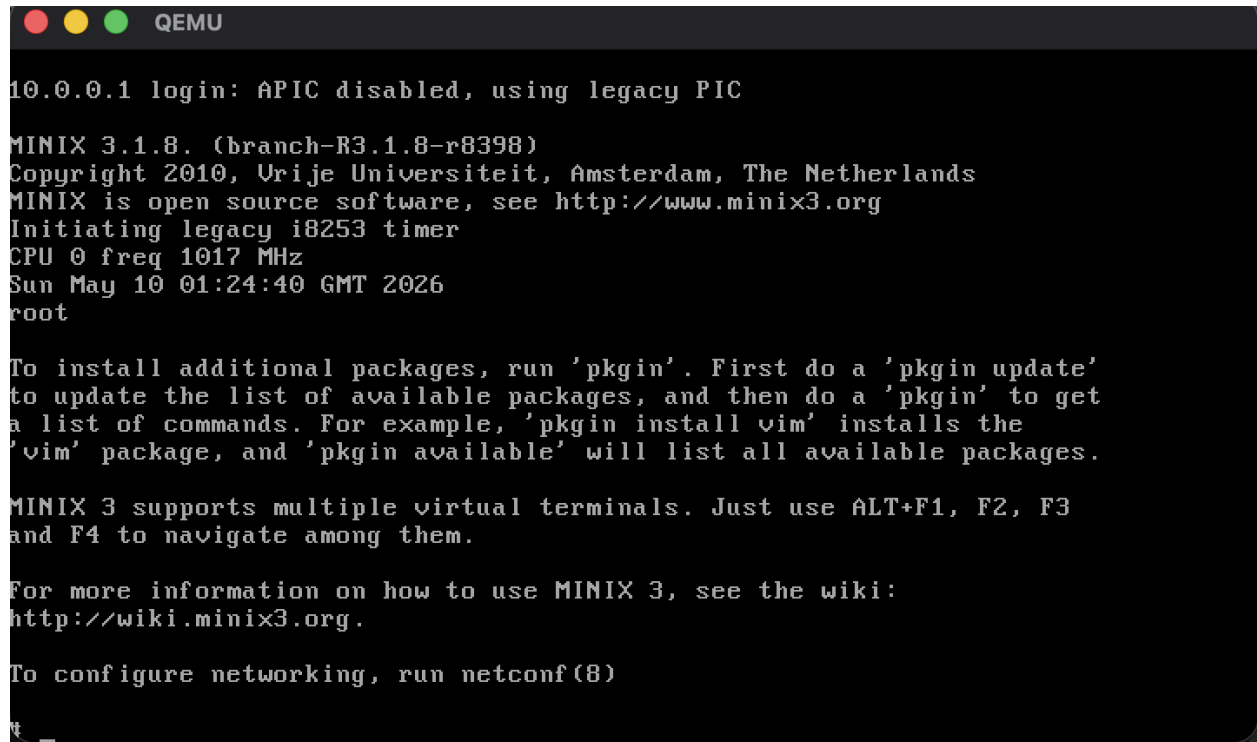
To configure networking, run netconf(8)

# shutdown@

Broadcast message from root@10.0.0.1 (console)
Sun May 10 01:24:02 2026...

The system will shutdown NOW

Local packages (down): sshd  done.
Sending SIGTERM to all processes ...
MINIX will now be shut down ...
ld0p0s0>image=/boot/image/3.1.8
ld0p0s0>save
ld0p0s0>_
```

A screenshot of a QEMU terminal window. The title bar shows three colored circles (red, yellow, green) and the text 'QEMU'. The terminal output shows the MINIX 3.1.8 login screen. It starts with '10.0.0.1 login: APIC disabled, using legacy PIC'. Then it displays 'MINIX 3.1.8. (branch-R3.1.8-r8398)', 'Copyright 2010, Vrije Universiteit, Amsterdam, The Netherlands', 'MINIX is open source software, see http://www.minix3.org', 'Initiating legacy i8253 timer', 'CPU 0 freq 1017 MHz', 'Sun May 10 01:24:40 GMT 2026', and 'root'. Below this, there is a paragraph explaining how to use 'pkgin' to install and update packages. Another paragraph mentions that MINIX 3 supports multiple virtual terminals and can be navigated using ALT+F1, F2, F3, and F4. A third paragraph provides a link to the MINIX 3 wiki for more information. The last line says 'To configure networking, run netconf(8)'. The prompt character is a dollar sign '\$'.

Problems Encountered:

N/A.

Solution:

N/A.

Lessons Learned:

I learned that MINIX keeps multiple boot images, and the boot monitor can choose which one to boot. This is useful when a modified kernel works but should not stay as the default.